

1.

```
import statistics
```

```
def calculate_mean(data):
```

```
    return sum(data) / len(data)
```

```
def calculate_median(data):
```

```
    sorted_data = sorted(data) n = len(sorted_data)
```

```
    if n % 2 == 0:
```

```
        middle1 = sorted_data[n // 2 - 1]
```

```
        middle2 = sorted_data[n // 2]
```

```
        return (middle1 + middle2) / 2
```

```
    else:
```

```
        return sorted_data[n // 2]
```

```
def calculate_mode(data):
```

```
    return statistics.mode(data)
```

```
def calculate_variance(data):
```

```
    mean_value = calculate_mean(data)
```

```
    squared_diff_sum = sum((x - mean_value) ** 2 for x in data)
```

```
    return squared_diff_sum / (len(data) - 1)
```

```
def calculate_standard_deviation(data):
```

```
    variance_value = calculate_variance(data)
```

```
    return variance_value ** 0.5
```

```
dataset = [10, 20, 30, 40, 50]
```

```
mean_value = calculate_mean(dataset)
```

```
median_value = calculate_median(dataset)
```

```
mode_value = calculate_mode(dataset)
```

```
variance_value = calculate_variance(dataset)

std_deviation_value = calculate_standard_deviation(dataset)

print(f'Dataset: {dataset}')

print(f'Mean: {mean_value:.2f}')

print(f'Median: {median_value:.2f}')

print(f'Mode: {mode_value}')

print(f'Variance: {variance_value:.2f}')

print(f'Standard Deviation: {std_deviation_value:.2f}')
```

## OUTPUT:

```
Dataset: [10, 20, 30, 40, 50]
Mean: 30.00
Median: 30.00
Mode: 10
Variance: 250.00
Standard Deviation: 15.81
```

---

2.

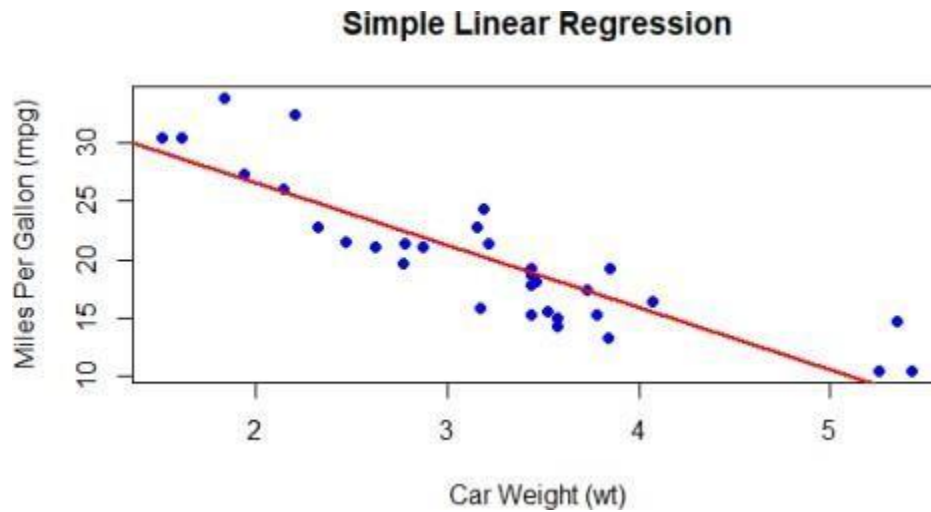
### Linear regression

```
data(mtcars)
model_simple <- lm(mpg ~ wt, data = mtcars)
pred_simple <- predict(model_simple, newdata = data.frame(wt = c(2.5, 3.0, 3.5)))
print(pred_simple)
plot(mtcars$wt, mtcars$mpg,
     main = "Simple Linear Regression",
     xlab = "Car Weight (wt)",
     ylab = "Miles Per Gallon (mpg)",
     pch = 19,
```

```
col = "blue")  
abline(model_simple, col = "red", lwd = 2)
```

### Output:

```
1      2      3  
23.92395 21.25171 18.57948
```



### Multiple regression:

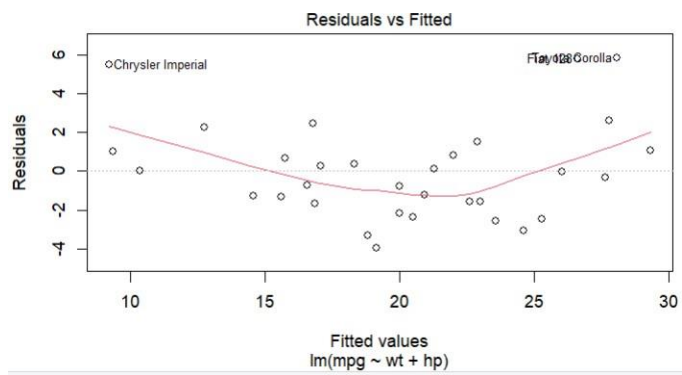
```
data(mtcars)  
  
model_multiple <- lm(mpg ~ wt + hp, data = mtcars)  
  
pred_multiple <- predict(model_multiple, newdata = data.frame(wt = c(3, 4), hp = c(110, 150)))  
  
print(pred_multiple)
```

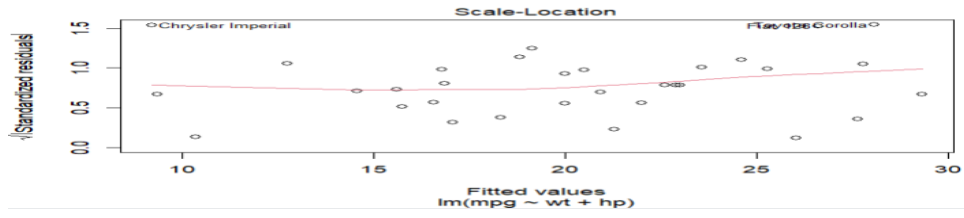
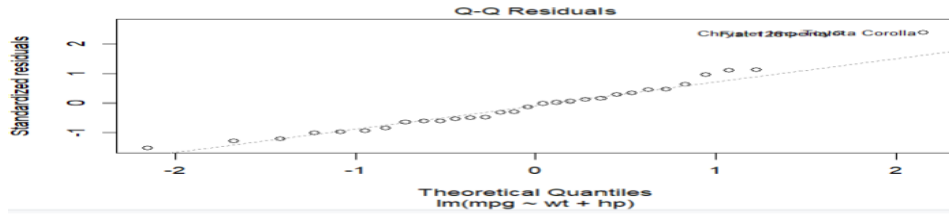
```
plot(model_multiple)
```

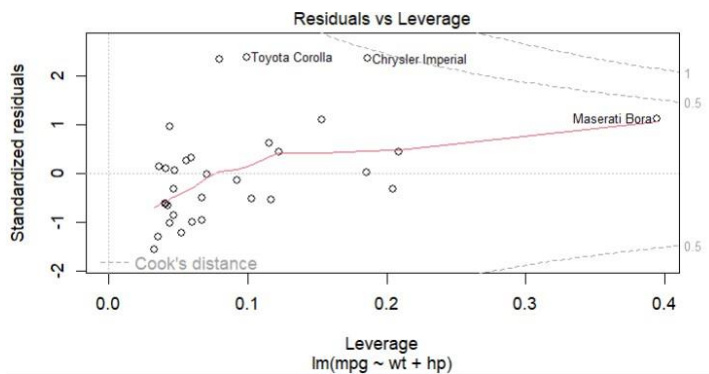
**OUTPUT:**

1 2

22.09875 16.95001







### 3. LOG\_REG

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score, confusion_matrix
iris = load_iris()
```

```
X = iris.data
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42
```

```
)
```

```
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
print("Actual Values:")
print(y_test)
```

```
print("\nPredicted Values:")
print(y_pred)
print("\nAccuracy:")
```

```
print(accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

## **OUTPUT:**

Actual Values:

```
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
```

Predicted Values:

[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0]

Accuracy:  
1.0

Confusion Matrix:

[[10 0 0]  
[ 0 9 0]  
[ 0 0 11]]



#### 4. Bin\_cla

```
library(caret)
```

```
library(dplyr)
```

```
library(ggplot2)
```

```
data(iris)
```

```
iris$Species <- factor(iris$Species == "setosa")
```

```
set.seed(123)
```

```
index <- createDataPartition(iris$Species, p = 0.7, list = FALSE)
```

```
train_data <- iris[index, ]
```

```

test_data <- iris[-index, ]

model <- glm(Species ~ ., data = train_data, family = binomial)

pred <- predict(model, newdata = test_data, type = "response")
pred_class <- ifelse(pred > 0.5, 1, 0)

cm <- table(Actual = test_data$Species, Predicted = pred_class)
print(cm)

accuracy <- sum(diag(cm)) / sum(cm)
print(accuracy)

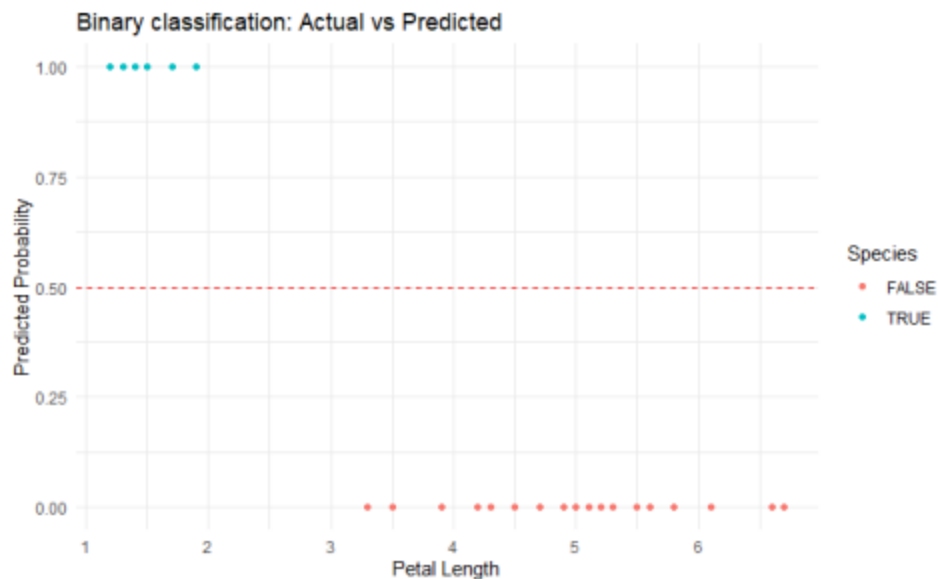
ggplot(test_data, aes(Petal.Length, pred, color = Species)) +
  geom_point() +
  geom_hline(yintercept = 0.5, linetype = "dashed", color = "red")

```

### Output:

|        | Predicted |    |
|--------|-----------|----|
| Actual | 0         | 1  |
| FALSE  | 30        | 0  |
| TRUE   | 0         | 15 |

```
[1] 1
```



## 5. knn\_nav

```
library(class)
library(e1071)
library(caret)
library(ggplot2)
```

```
data(iris)
```

```
set.seed(123)
```

```
train <- createDataPartition(iris$Species, p = 0.7, list = FALSE)
```

```
train_data <- iris[train, ]
```

```
test_data <- iris[-train, ]
```

```
knn_result <- knn(train_data[,1:4], test_data[,1:4], train_data$Species, k = 3)
```

```
cat("KNN Confusion Matrix\n")
```

```
print(table(knn_result, test_data$Species))
```

```
cat("KNN Accuracy =", mean(knn_result == test_data$Species), "\n")
```

```
ggplot(test_data, aes(Petal.Length, Petal.Width, color = knn_result)) +
```

```

geom_point(size = 3) +
labs(title = "KNN Classification") +
theme_minimal()

nb_model <- naiveBayes(Species ~ ., data = train_data)
nb_result <- predict(nb_model, test_data)

cat("Naive Bayes Confusion Matrix\n")
print(table(nb_result, test_data$Species))
cat("Naive Bayes Accuracy =", mean(nb_result == test_data$Species), "\n")

ggplot(test_data, aes(Petal.Length, Petal.Width, color = nb_result)) +
  geom_point(size = 3) +
  labs(title = "Naive Bayes Classification") +
  theme_minimal()

```

### Output:

KNN Confusion Matrix

|            | test_data\$Species |            |           |
|------------|--------------------|------------|-----------|
| knn_result | setosa             | versicolor | virginica |
| setosa     | 15                 | 0          | 0         |
| versicolor | 0                  | 15         | 1         |
| virginica  | 0                  | 0          | 14        |

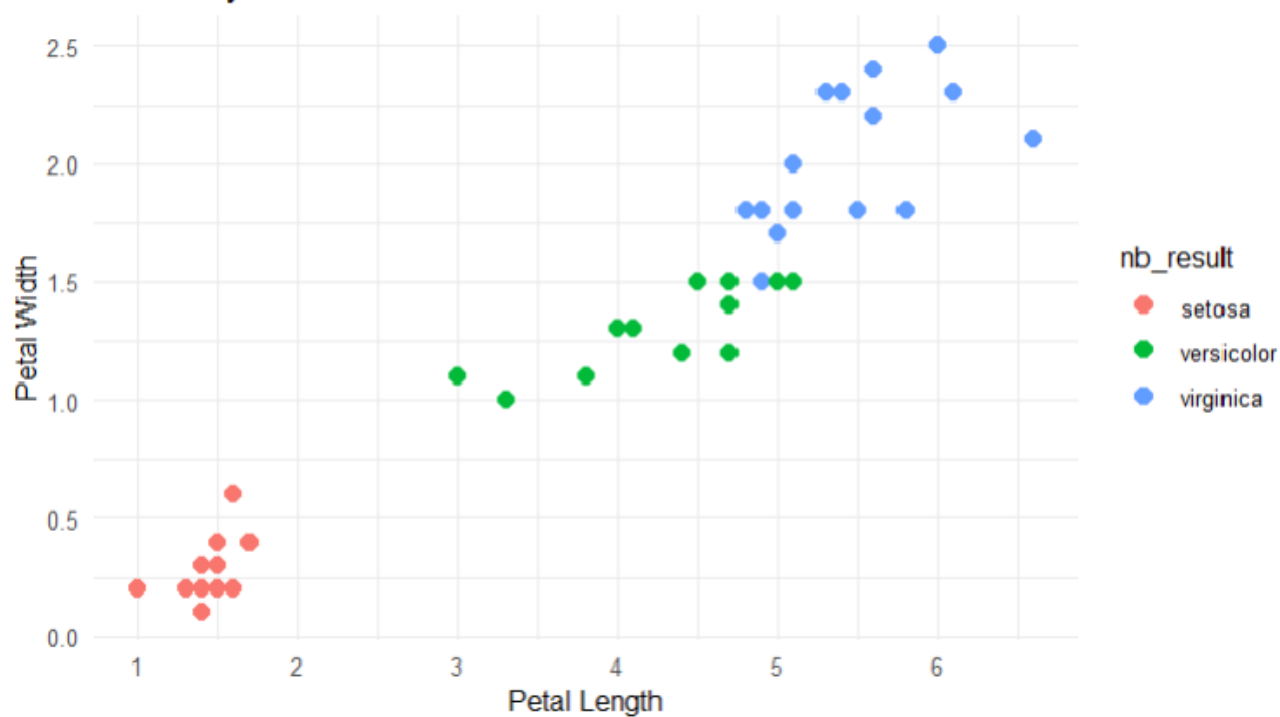
KNN Accuracy = 0.977778

Naive Bayes Confusion Matrix

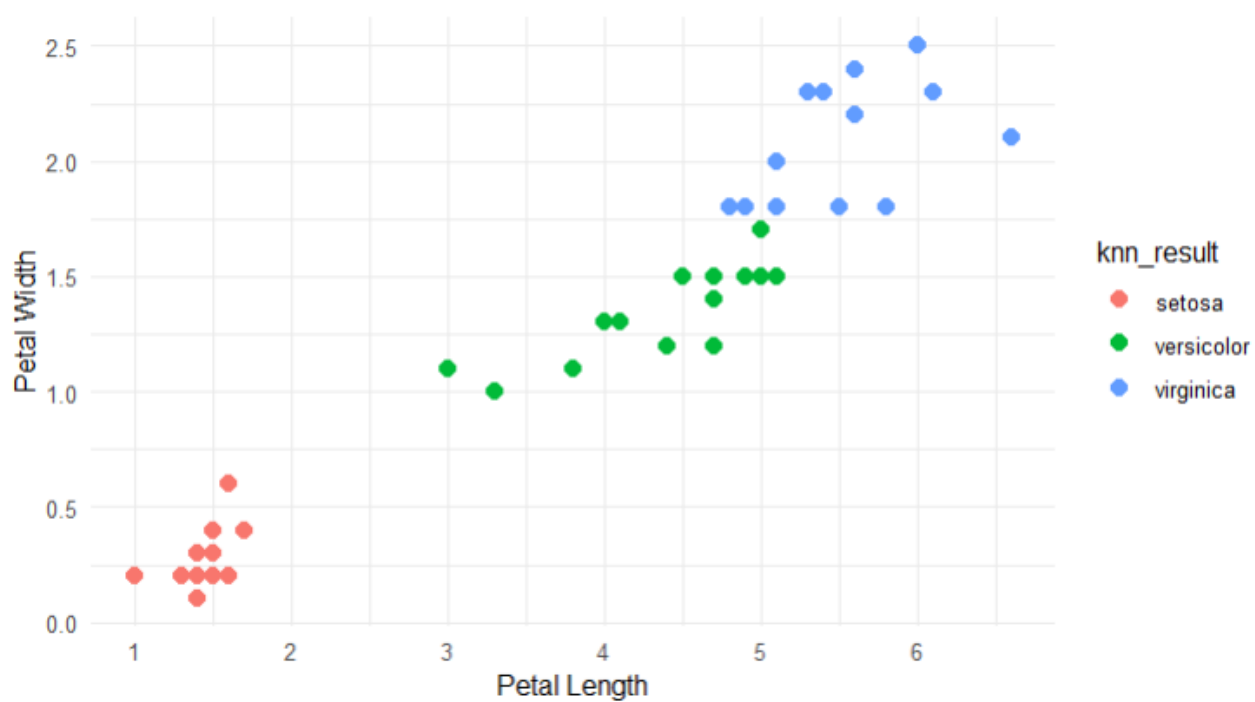
|            | test_data\$Species |            |           |
|------------|--------------------|------------|-----------|
| nb_result  | setosa             | versicolor | virginica |
| setosa     | 15                 | 0          | 0         |
| versicolor | 0                  | 15         | 1         |
| virginica  | 0                  | 0          | 14        |

Naive Bayes Accuracy = 0.977778

Naive Bayes classification



KNN classification



```
data(iris)
iris_data <- iris[, 1:4]
set.seed(123)
kmeans_result <- kmeans(iris_data, centers = 3)
print(kmeans_result)
iris$cluster <- factor(kmeans_result$cluster)
library(ggplot2)
ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, color = cluster))
  + geom_point(size = 3)
  + labs(title = "K-Means Clustering of Iris Dataset", x = "Sepal Length", y =
"Sepal Width")
  + theme_minimal()
```

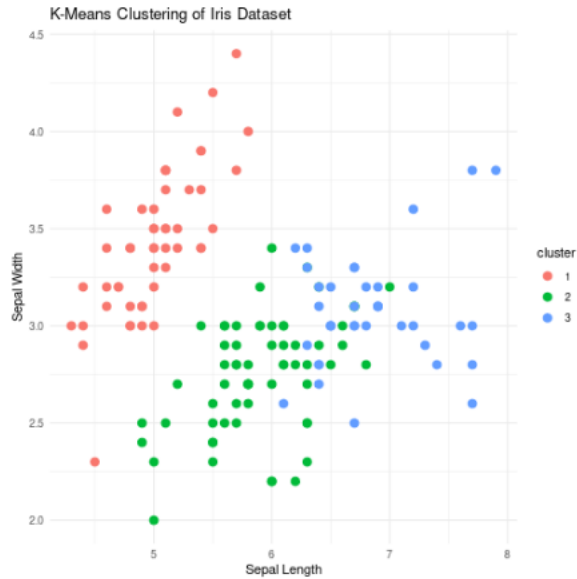
Cluster means:

Clustering vector:

[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1  
[38] 1 1 1 1 1 1 1 1 1 1 1 1 2 2 3 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2  
[75] 2 2 2 3 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 2 3 3 3 2 3 3 3  
[112] 3 3 2 2 3 3 3 3 2 3 2 3 2 3 3 2 3 3 3 3 2 3 3 3 2 3 3 3 2 3  
[149] 3 2

[1] 15.15100 39.82097 23.87947  
(between SS / total SS = 88.4 %)

```
[1] "cluster"    "centers"    "totss"      "withinss"   "tot.withinss"
[6] "betweenss"  "size"       "iter"       "ifault"
```



8.

```
install.packages("tensorflow")
```

```
Install.packages("keras")
```

```
system("apt-get update -qq")
```

```
system("apt-get install -y python3.12-venv python3-pip")
```

```
library(tensorflow)
```

```
install_tensorflow()
```

```
tf$constant("Hello TensorFlow")
```

```
Library(keras)
```

```
install.packages("keras3")
```

```
library(keras3)
```

```
library(keras3)
```

```
x_train <- array(runif(1000 * 30 * 30 * 3), dim = c(1000, 30, 30, 3))
```

```
y_train <- sample(0:9, 1000, replace = TRUE)
```

```
x_test <- array(runif(200 * 30 * 30 * 3), dim = c(200, 30, 30, 3))
```

```
y_test <- sample(0:9, 200, replace = TRUE)
```

```
model <- keras_model_sequential(input_shape = c(30, 30, 3)) |>
```

```
  layer_conv_2d(filters = 16, kernel_size = c(3, 3), activation = "relu") |>
```

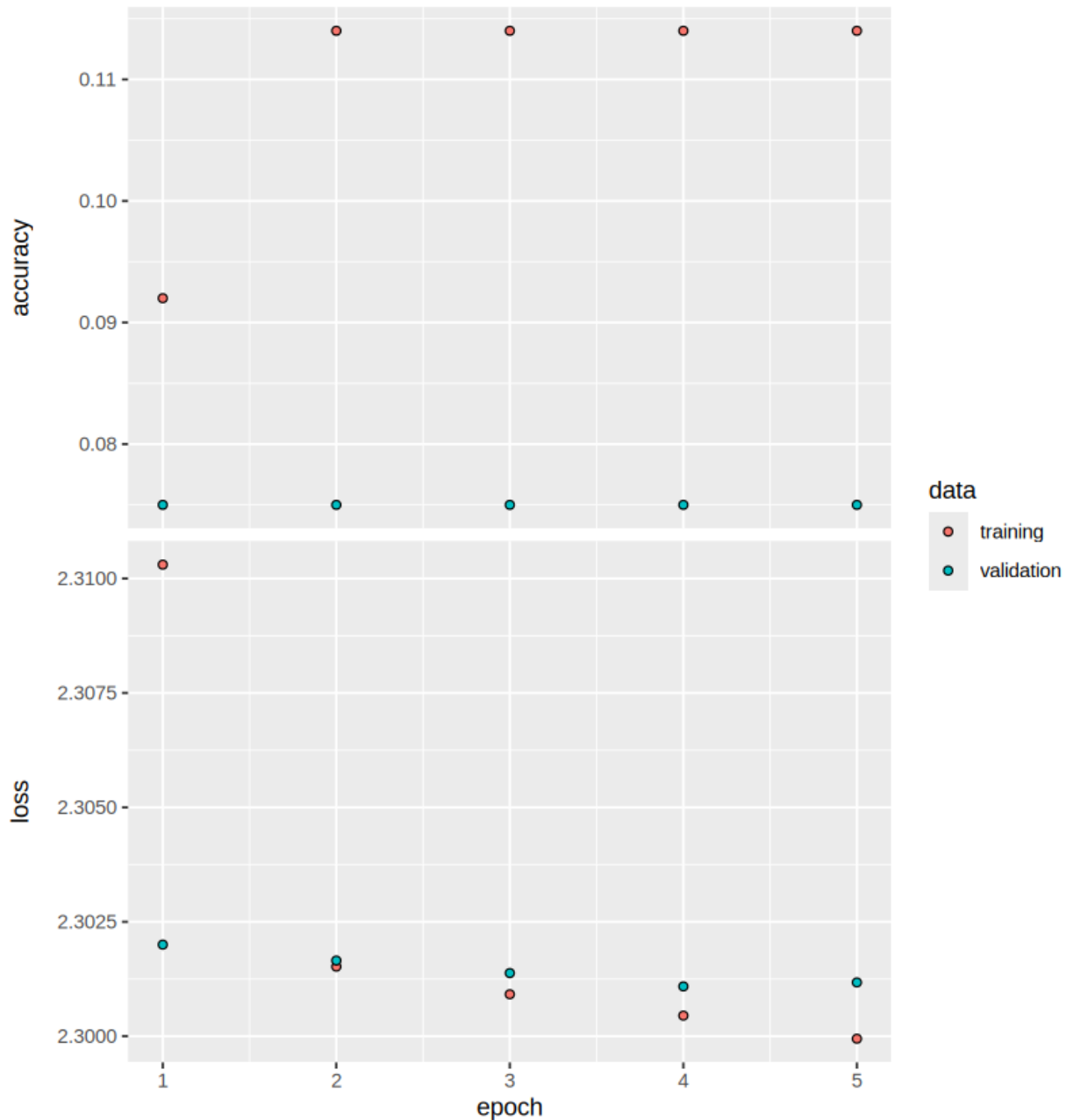
```
layer_max_pooling_2d(pool_size = c(2, 2)) |>  
layer_flatten() |>  
layer_dense(units = 32, activation = "relu") |>  
layer_dense(units = 10, activation = "softmax")
```

```
model |> compile(  
  optimizer = "adam",  
  loss = "sparse_categorical_crossentropy",  
  metrics = "accuracy"  
)
```

```
history <- model |> fit(  
  x_train,  
  y_train,  
  epochs = 5,  
  batch_size = 30,  
  validation_data = list(x_test, y_test),  
  verbose = 1  
)
```

```
plot(history)
```





9. library(keras3)

set.seed(123)

data <- scale(data.frame(x1 = rnorm(200, 3, 1), x2 = rnorm(200, 5, 1)))

# 2. Build Keras Autoencoder (2D -> 1D -> 2D)

input\_layer <- layer\_input(shape = 2)

encoded <- layer\_dense(input\_layer, units = 1, activation = 'relu')

decoded <- layer\_dense(encoded, units = 2, activation = 'linear')

autoencoder <- keras\_model(input\_layer, decoded)

# 3. Train and Predict

```

autoencoder %>% compile(optimizer = 'adam', loss = 'mse')
autoencoder %>% fit(as.matrix(data), as.matrix(data), epochs = 100, batch_size = 10,
verbose = 0)
reconstructed <- autoencoder %>% predict(as.matrix(data))

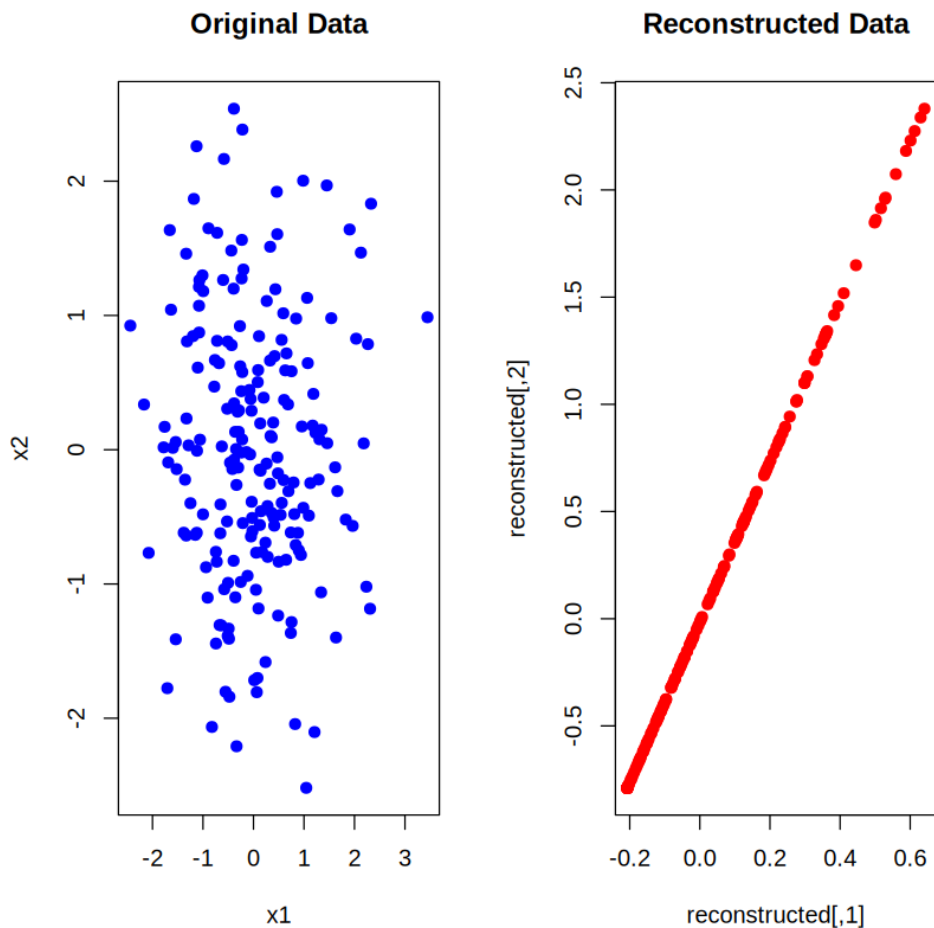
```

#### # 4. Generate the Comparison Diagram

```

par(mfrow = c(1, 2))
plot(data, main = 'Original Data', col = 'blue', pch = 19)
plot(reconstructed, main = 'Reconstructed Data', col = 'red', pch = 19)

```



10.

```
library(keras3)
```

```

x <- array(runif(100 * 5), dim = c(100, 5, 1))
y <- runif(100)

```

```
model <- keras_model_sequential() |>
```

```
layer_lstm(units = 10, input_shape = c(5, 1)) |>  
layer_dense(units = 1)
```

```
model |> compile(  
  optimizer = "adam",  
  loss = "mse"  
)
```

```
model |> fit(  
  x, y,  
  epochs = 10,  
  batch_size = 10,  
  verbose = 0  
)
```

```
prediction <- model |> predict(x)
```

```
plot(  
  y,  
  type = "l",  
  col = "blue",  
  main = "LSTM Prediction",  
  xlab = "Sample",  
  ylab = "Value"  
)
```

```
lines(  
  prediction,  
  col = "red",  
  lwd = 2  
)
```

```
legend(  
  "topright",  
  legend = c("Actual", "Predicted"),  
  col = c("blue", "red"),  
  lty = 1
```

)

